

Greenplum

Technical Publications

Greenplum Performance Monitor Guide 2.0.0

Last Revised: October 24, 2008 4:19 pm

Contents - Greenplum Performance Monitor Guide

About This Guide 1
Greenplum Database Documentation 1
Document Conventions 1
Contact Us 4

Chapter 1: About Greenplum Performance Monitor 5
Greenplum Performance Monitor Architecture 5
Greenplum Performance Monitor Agents 6
Greenplum Performance Monitor Database 6
Current Data Tables 6
Tail Data Tables 7
Historical Data Tables 7
Greenplum Performance Monitor User Interface 7
Greenplum Performance Monitor Web Server 7

Chapter 2: Installing Greenplum Performance Monitor 8
Preparing for Installation 8
System Requirements 8
Creating the Monitor Superuser 8
Editing postgresql.conf 9
Installing Greenplum Performance Monitor on the Master 9
Installing Greenplum Performance Monitor on the Master 9
Setting up Greenplum Performance Monitor on the Master 10
Installing the Segment Monitor Agents 12
Uninstalling Greenplum Performance Monitor 12

Chapter 3: Administering Greenplum Performance Monitor 14
Starting and Stopping Greenplum Performance Monitor 14
Reconfiguring Greenplum Performance Monitor 14
Editing the Configuration File 14
Reconfiguring a Monitor Instance with gpperfmon 15
Agent Administration 16
Restarting the Master Agent 16
Adding and Removing Monitored Hosts 16
Viewing and Maintaining Master Agent Log Files 16
Greenplum Performance Monitor Database Administration 17
Backing Up and Restoring the Monitor Database 17
Maintaining Historical Data 17
Viewing and Maintaining Database Log Files 17
Web Server Administration 17
Starting and Stopping the Web Server 17
Viewing and Maintaining Web Server Log Files 18

Chapter 4: Using Greenplum Performance Monitor 19
About the Greenplum Performance Monitor User Interface 19
Monitoring the Current System Workload 19
Monitoring Active Queries 19
Viewing Historical Query Metrics 19

Appendix A: Utility Reference: gpperfmon 20

Appendix B: Configuration Reference: gpperfmon.conf 22
Master Agent Options 22
User Interface Options 23

Appendix C: Greenplum Performance Monitor Catalog Reference 25
system_now 26
system_tail 28
system_history 30
database_now 32
database_tail 33
database_history 34
queries_now 35
queries_tail 36
queries_history 37
iterators_now 38
iterators_tail 43
iterators_history 48

Copyright © 2008 by Greenplum, Inc. All rights reserved.

This publication pertains to Greenplum software and to any subsequent release until otherwise indicated in new editions or technical notes.

Information in this document is subject to change without notice. The software described herein is furnished under a license agreement, and it may be used or copied only in accordance with the terms of that agreement. Upgrades are provided only at regularly scheduled software release dates. No part of this publication may be reproduced, transmitted, or translated in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without the prior written permission of Greenplum, Inc. Greenplum makes no warranty of any kind with respect to the completeness or accuracy of this manual.

Greenplum, the Greenplum logo, and Greenplum Database are trademarks of Greenplum, Inc. PostgreSQL is a trademark of Marc Fournier held in trust for The PostgreSQL Development Group. All other company and product names used herein may be trademarks or registered trademarks of their respective companies.

Some Greenplum Database management tools make use of the Apache Portable Runtime library (http://www.apache.org/licenses) and the Eventlib library Copyright 2000-2004 Niels Provos and are subject to their respective license agreements.

Preface

This guide provides information for system administrators and Greenplum Database super users responsible for administering the Greenplum Performance Monitor.

- About This Guide
- Greenplum Database Documentation
- Contact Us

About This Guide

This guide provides information and instructions for installing, configuring and maintaining Greenplum Performance Monitor. This guide is intended for system administrators and database super users responsible for managing the Greenplum system and databases.

This guide assumes knowledge of Linux/Unix system administration, database management systems, database administration, and structured query language (SQL).

Greenplum Database Documentation

The Greenplum Database documentation is provided in PDF and HTML format. The documentation can be found in the `$GPHOME/docs` directory of your Greenplum Database installation, or on the Greenplum Network.

The Greenplum Database documentation is intended as a supplement to the PostgreSQL 8.2 documentation, upon which Greenplum Database is based.

The following documents are provided with this release of Greenplum Database:

- **Release Notes** — The release notes (`README`) explain the new features and any known issues associated with this release.
- **Greenplum Database Administrator Guide** — This guide contains conceptual, instructional and reference information for database and system administrators responsible for installing, configuring and administering a Greenplum Database system.

Document Conventions

The following conventions are used throughout the Greenplum Database documentation to help you identify certain types of information.

Text Conventions

Table 0.1 Text Conventions

Text Convention	Usage	Examples
bold	Button, menu, tab, page, and field names in GUI applications	Click **Cancel** to exit the page without saving your changes.
italics	New terms where they are defined Database objects, such as schema, table, or columns names	The *master instance* is the postmaster process that accepts client connections. Catalog information for Greenplum Database resides in the *pg_catalog* schema.
`monospace`	File names and path names Programs and executables Command names and syntax Parameter names	Edit the `postgresql.conf` file. Use `gpstart` to start Greenplum Database.
`monospace italics`	Variable information within file paths and file names Variable information within command syntax	`/home/gpadmin/`*`config_file`* `COPY` *`tablename`* `FROM '`*`filename`*`'`
`monospace bold`	Used to call attention to a particular part of a command, parameter, or code snippet.	Change the host name, port, and database name in the JDBC connection URL: `jdbc:postgresql://`**`host:5432/mydb`**
`UPPERCASE`	Environment variables SQL commands Keyboard keys	Make sure that the Java `/bin` directory is in your `$PATH`. `SELECT * FROM my_table;` Press `CTRL+C` to escape.

Command Syntax Conventions

Table 0.2 Command Syntax Conventions

Text Convention	Usage	Examples
`{ }`	Within command syntax, curly braces group related command options. Do not type the curly braces.	`FROM { 'filename' | STDIN }`
`[ ]`	Within command syntax, square brackets denote optional arguments. Do not type the brackets.	`TRUNCATE [ TABLE ] name`
`...`	Within command syntax, an ellipsis denotes repetition of a command, variable, or option. Do not type the ellipsis.	`DROP TABLE name [, ...]`
`|`	Within command syntax, the pipe symbol denotes an "OR" relationship. Do not type the pipe symbol.	`VACUUM [ FULL | FREEZE ]`

`$ system_command`
`# root_system_command`
`=> gpdb_command`
`=# su_gpdb_command`
	Denotes a command prompt - do not type the prompt symbol. `$` and `#` denote terminal command prompts. `=>` and `=#` denote Greenplum Database interactive program command prompts (`psql` or `gpssh`, for example).	`$ createdb mydatabase`
		`# chown gpadmin -R /datadir`
		`=> SELECT * FROM mytable;`
		`=# SELECT * FROM pg_database;`

Naming Conventions and Acronyms

Table 0.3 Naming Conventions and Acronyms

Convention	Meaning
`$GPHOME`	The base directory where Greenplum Database is installed, for example: `/usr/local/greenplum-db-gpmon-3.2.0.0`
`gpadmin`	The default name for the Greenplum Database super user.
`KB`	Kilobytes (2^{10})
`MB`	Megabytes (2^{20})
`GB`	Gigabytes (2^{30})
`TB`	Terabytes (2^{40})

Contact Us

To contact Greenplum customer support, call 1-866-410-6060 or send email to:

support@greenplum.com

1 About Greenplum Performance Monitor

Greenplum provides an optional performance monitoring feature that administrators can install and enable for Greenplum Database. Greenplum Performance Monitor lets administrators view the status and performance of actively running queries and allows for detailed analysis of historical queries and system metrics. A browser-based graphical interface provides easy access to a variety of configurable views and reports on performance data.

Greenplum Performance Monitor Architecture

Greenplum Performance Monitor is based on an agent architecture. Agent processes are installed on each segment host, and all other monitor components are installed on the master host.

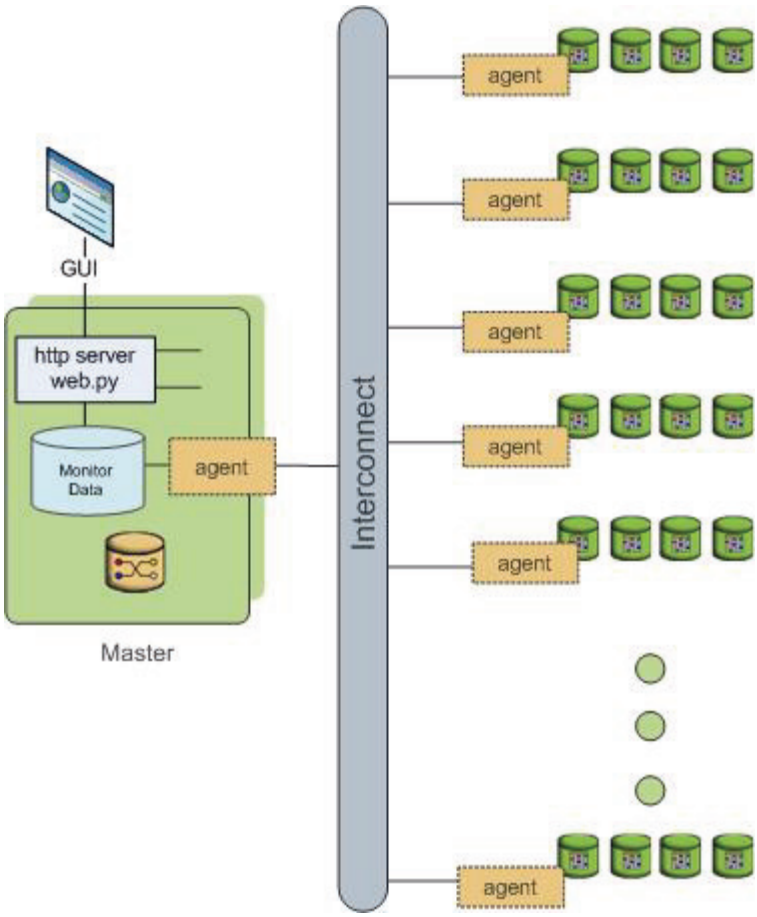


Figure 1.1 Greenplum Performance Monitor Architecture

The agents collect performance data about active queries and system utilization and send the data to the Greenplum master at regular intervals. The data is stored in a dedicated Greenplum Performance Monitor database on the master, where it can be accessed through the user interface or through SQL queries.

Greenplum Performance Monitor Agents

Greenplum Performance Monitor agents run on all hosts in a Greenplum Database array. Each segment host contains its own segment agent `gpsmon` process, which collects runtime statistics for system metrics, queries, and iterators. A master agent `gpmmon` process controls and maintains TCP connections to the segment agents.

At runtime, segment agent processes send performance data to the master agent at configurable intervals called `quantum`. The master agent pulls together the data from all agents, stores it in flat files, and periodically commits the data in the files to the Greenplum Performance Monitor database.

Greenplum Performance Monitor Database

The Greenplum Performance Monitor database is a distributed Greenplum database dedicated to storing and serving performance data for the system. The `gpperfmon` installation utility runs SQL cripts to build and initialize this database on the master host.

Greenplum administrators can connect to the monitor database to manage and query performance data using client programs such as `psql` or application programming interfaces (APIs) such as JDBC and ODBC. Also, administrators can view reports on current and historical peformance data in the Greenplum Performance Monitor graphical interface.

The database consists of three sets of tables, one set for current data, a “tail” set for data in transition, and another set for historical data.

Table 1.1 Greenplum Performance Monitor Data Tables

Current Data Table (External)	Tail Data Table (External)	Historical Data Table
system_now	*system_tail*	*system_history*
database_now	*database_tail*	*database_history*
queries_now	*queries_tail*	*queries_history*
iterators_now	*iterators_tail*	*iterators_history*

Current Data Tables

The current data is stored in flat files that are treated as external tables by the database. These files are located in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`.

For metrics, a row of data persists in the `metrics_now` table for one quantum period, after which it is replaced by the next row for that period. Query and iterator data persists in the current data tables until the query is completed, refreshing every quantum period.

Tail Data Tables

After passing out of the current data tables, all data is stored in the tail tables for a fixed period of <TBD: at time of writing> two minutes before being committed to the historical data tables. These files are located in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`.



Note: <TBD: Inside `$MASTER_DATA_DIRECTORY/gpperfmon/data/` you may see temporary files with the prefix `_tail`. Do not query or manipulate these unstable files, which may be safely ignored.>

Historical Data Tables

The history tables are partitioned by month, allowing you to drop partitions for data that you no longer need. The `gpperfmon` installer creates eighteen months of partitioned tables. For more information on these tables and the Greenplum Performance Monitor database schema, see Appendix C, “Greenplum Performance Monitor Catalog Reference”.

Greenplum Performance Monitor User Interface

Greenplum Performance Monitor provides a graphical interface for viewing performance metrics. With this browser-based application, you can create customized graphical views for tracking the performance of active queries and for analyzing historical data for queries and system metrics.

<TBD: screen shot of GUI with callouts>

For more information, see Chapter 4, “Using Greenplum Performance Monitor”.

Greenplum Performance Monitor Web Server

The Greenplum Performance Monitor user interface is powered by a lightweight lighttpd web server. The lighttpd server is an open-source web server with a low memory footprint and light CPU load. For more information, see http://www.lighttpd.net/.

The `gpperfmon` installer sets up the lighttpd web server, prompting you for basic configuration information on the desired port and SSL options. Under normal conditions, the web server requires minimal maintenance and administration, as described in “Web Server Administration” on page 17.

<TBD: create a“follow a query through the monitor” example or graphic? Would it go here or up with the architecture diagram? >

2 Installing Greenplum Performance Monitor

Installing Greenplum Performance Monitor involves the following high-level steps:

- Preparing for Installation
- Installing Greenplum Performance Monitor on the Master
- Installing the Segment Monitor Agents

This chapter also provides information about uninstalling Greenplum Performance Monitor.

Preparing for Installation

Before installing Greenplum Performance Monitor, verify that your system meets the basic requirements, create a monitor superuser, and change settings in `postgresql.conf`.

You must be the Greenplum superuser (`gpadmin`) to install Greenplum Performance Monitor.

System Requirements

<TBD: Requirements for the GP master host amply accomodate the monitor's needs. How much of the master's RAM, CPU and disk will be used by the monitor and web server?>

Creating the Monitor Superuser

The Greenplum Performance Monitor requires a dedicated superuser role with database connection privileges. The monitor superuser owns the database that stores performance statistics data, connects to segment hosts to control agents, and performs other monitor-related operations.

To create the monitor superuser and assign connection privileges:

1. Add a new user with the name `gpmon`. For example:

   ```
   # useradd gpmon
   ```

2. Using `psql` or another PostgreSQL-compatible client program, create a new superuser role with the `CREATEDB` attribute. For example:

   ```
   $ psql template1 -c 'CREATE ROLE gpmon WITH SUPERUSER
   CREATEDB LOGIN;'
   ```

3. Edit `$MASTER_DATA_DIRECTORY/pg_hba.conf` to give the `gpmon` superuser privileges to connect to the database. For example, to allow trusted local connections only, add the following entries to the file:

   ```
   local     all     gpmon     trust
   host      all     gpmon     0.0.0.0/0 trust
   ```

4. Use `gpstop` with the `-u` option to reload `pg_hba.conf`:

   ```
   $ gpstop -u
   ```

Editing postgresql.conf

To enable the monitor in your Greenplum environment, you must change and verify `postgresql.conf` settings on all hosts as described in this section.

To configure Greenplum Performance Monitor:

1. <TBD: get the process to do this with gpssh> Edit `$MASTER_DATA_DIRECTORY/postgresql.conf` and set the following parameter to "on" as shown<TBD at time of writing, this must be added>:

   ```
   gp_enable_gpperfmon=on
   ```

2. Find the following parameter in the "EXTERNAL TABLES" section of `postgresql.conf` and verify that it is set to "on" as shown:

   ```
   gp_external_enable_exec=on
   ```

3. If you want to use a different port than the default port 8888, set the following parameter<TBD at time of writing, this must be added>:

   ```
   gpperfmon_port=<port_number>
   ```

4. Use `gpstop` with the `-u` option to reload `postgresql.conf`:

   ```
   $ gpstop -u
   ```



Note: The parameter `gp_enable_gpperfmon` is required for the collection of database, query and iterator data. The monitor can collect only system metrics when this parameter is not enabled.

Installing Greenplum Performance Monitor on the Master

Installing the monitor on the master host involves the following high-level tasks:

- Installing Greenplum Performance Monitor on the Master creates the installation directory on the host machine.
- Setting up Greenplum Performance Monitor on the Master creates and configures the the agents, the database, and other monitor components on the host machine.

Installing Greenplum Performance Monitor on the Master

This section describes how to create the installation directory for Greenplum Performance Monitor on your master host.

To install Greenplum Performance Monitor

1. Download or copy the installer file to the Greenplum Database master host. Installer files are available from Greenplum for RedHat (32-bit and 64-bit) and Solaris (64-bit) platforms.

2. Unzip the installer file where *PLATFORM* is either `RHEL4-i386` (RedHat 32-bit), `RHEL4-x86_64` (RedHat 64-bit) or `SOL-x86_64` (Solaris 64-bit). For example:

   ```
   # unzip greenplum-perfmon-2.0.0-PLATFORM.zip
   ```

3. Launch the installer using `bash`. For example:

   ```
   # /bin/bash greenplum-perfmon-2.0.0-PLATFORM.bin
   ```

4. The installer will prompt you to accept the Greenplum Performance Monitor license agreement. Type `yes` to accept the license agreement.

5. The installer will prompt you to provide an installation path. Press `ENTER` to accept the default install path (`/usr/local/greenplum-perfmon-2.0.0`), or enter an absolute path to an install location. You must have write permissions to the location you specify.

6. The installation directory contains the file `gpperfmon_path.sh` with path and environment settings for the monitor. You must source this file in this directory, or the `gpadmin` user's startup shell profile (such as `.bashrc`).

   For example, you could add a line similar to the following to your chosen profile files:

   ```
   source /usr/local/greenplum-perfmon-2.0.0/gpperfmon_path.sh
   ```

   After editing the chosen profile file, source it as the correct user to make the changes active. For example:

   ```
   $ source ~/.bashrc
   ```

   After sourcing `gpperfmon_path.sh`, you can continue to Setting up Greenplum Performance Monitor on the Master.

Setting up Greenplum Performance Monitor on the Master

The `gpperfmon` utility sets up Greenplum Performance Monitor on the master. This utility creates the following components on the master host:

- Dedicated Greenplum database for performance data (`gpperfmon`)
- Master monitor agent (`gpmmon`)
- HTTP server and scripts to support the user interface

During the setup process, the utility prompts you for values to configure these components. For more information on choosing values for your environment, see detailed descriptions of the configuration parameters in Appendix B, "Configuration Reference: gpperfmon.conf"

To accept the displayed default values for any of these parameters at configuration time, hit the `ENTER` key.

To set up Greenplum performance monitor on the master

1. Launch `gpperfmon` with the `--setup` option. For example:

   ```
   $ gpperfmon --setup
   ```

2. The utility prompts you to create the Greenplum Performance Monitor database. Type `Y` to agree and then type `Y` again to verify. The installer runs SQL scripts to create the database and displays a message indicating that the database was created successfully.
3. Enter a value for Quantum. The default is 15 seconds.
4. Enter a value for Minimum query run time. The default is 60.
5. Enter a value for Minimum detailed query run time. The default value displayed is the value you specified for Minimum query run time.
6. Enter a location for the Greenplum Performance Monitor log files. The default is `$MASTER_DATA_DIRECTORY/gpperfmon/logs`.
7. Enter a value for the maximum number of data points the Web UI can return in one call. The default is 600.
8. The monitoring GUI can highlight specific query or system metrics that exceed a certain threshold. Enter desired values for the following metrics:

   ```
   Load2 threshold [2.00]:
   CPU usage threshold (percent) [80]:
   Memory usage threshold (percent) [80]:
   Disk usage threshold (bytes) [0]:
   Network usage threshold (bytes) [10000]:
   CPU elapsed threshold [70]:
   CPU skew threshold [100]:
   ```

9. Enter the web server port for the user interface. The default 8080.
10. (Optional) Enter `Y` or `N` to set up SSL connections for the user interface. If you enter `Y`, you are prompted for the following DN information:

    ```
    Country Name (2 letter code) [GB]:
    State or Province Name (full name) [Berkshire]:
    Locality Name (eg, city) [Newbury]:, s
    Organization Name (eg, company) [My Company Ltd]:
    Organizational Unit Name (eg, section) []:
    Common Name (eg, your name or your server's hostname) []:
    Email Address []:
    ```

    When the utility displays a message indicating that `gpperfmon` configuration is complete, you have successfully installed Greenplum Performance Monitor on your master host.

For more information on next steps, see Installing the Segment Monitor Agents and Starting and Stopping Greenplum Performance Monitor.

Installing the Segment Monitor Agents

All segment agents must be installed in the same directory location as the installation directory on the master. For instance, if you installed to the default directory on the master, you must install all segment monitor agents to `/usr/local/greenplum-perfmon-2.0.0`.

To install Monitor Agents on the Segment Hosts

1. On the master host, create a tar file of your Greenplum Performance Monitor installation. For example (running as `root`):

   ```
   # su -
   # cd /usr/local
   # gtar -cvf /usr/local/perfmon.tar greenplum-perfmon-2.0.0
   ```

2. Copy the tar file to the segment hosts using `gpscp`. For example:

   ```
   # gpscp -f /home/gpadmin/single_seg_hosts_file
   /usr/local/perfmon.tar =:/usr/local
   ```

3. Start an interactive session in `gpssh`. For example:

   ```
   # gpssh -f /home/gpadmin/single_seg_hosts_file
   ```

4. At the `gpssh` command prompt, untar the tar file in the installation directory on the segment hosts. This must be the same location as `gpperfmon` on your master host. For example:

   ```
   => gtar --directory /usr/local -xvf /usr/local/perfmon.tar
   ```

5. Confirm that the Greenplum Performance Monitor directory was installed in the correct location (the same location as `gpperfmon` on your master host). For example:

   ```
   => ls /usr/local/greenplum-perfmon-2.0.0
   ```

6. Change the ownership of the Greenplum Database install directory to the `gpadmin` user or group. For example:

   ```
   => chown -R gpadmin /usr/local/greenplum-perfmon-2.0.0
   => chgrp -R gpadmin /usr/local/greenplum-perfmon-2.0.0
   ```

7. Remove the tar file. For example:

   ```
   => rm /usr/local/gp.tar
   ```

   Once you have installed the master agent and all segment agents, you can start the monitor as descrbed in "Starting and Stopping Greenplum Performance Monitor" on page 14.

Uninstalling Greenplum Performance Monitor

To uninstall, you must remove monitor directories, drop the monitor database, and disable the monitor in your Greenplum Database configuration.

To uninstall Greenplum Performance Monitor

1. On the Greenplum Database master, open `postgresql.conf` and change the value of `gp_enable_gpperfmon` to `off`.
2. Reload the configuration file using the `gpstop` utility. For example:

   ```
   $gpstop -u
   ```
3. Remove the monitor installation directory. For example:

   ```
   $ rm -rf gpperfmon-3.2
   ```
4. Remove the directory that contains monitor data and configuration information. In a default setup this is `$MASTER_DATA_DIRECTORY/gpperfmon`. For example:

   ```
   $ rm -rf gpperfmon
   ```
5. Drop the monitor database using the `dropdb` utility. For example:

   ```
   $ dropdb --echo gpperfmon
   DROP DATABASE gpperfmon;
   DROP DATABASE
   ```
6. Remove segment agents from the segment hosts using `gpssh` <TBD: process for removing the segment agents>

3 Administering Greenplum Performance Monitor

Much of the routine administration of Greenplum Performance Monitor is performed using the `gpperfmon` utility. This utility enables basic tasks such as stopping and restarting the monitor.

Other tasks for administration of the user interface web server or the monitor database are performed with native tools or Greenplum utilities, as described in this chapter.

Starting and Stopping Greenplum Performance Monitor

The `gpperfmon` utility starts, stops and restarts Greenplum Performance Monitor with the following option flags:

```
--start
--stop
--restart
```

For example, to start Greenplum Performance Monitor, run the following command:

```
$gpperfmon --start
```

These command options control the master agent, the segment agent processes, and the UI web server. If you want to exclude the UI web server from these commands, add the flag `--noweb`. For example, to restart agents without restarting the web server, run the following command:

```
$gpperfmon --restart --noweb
```

For more information about `gpperfmon`, see "Utility Reference: gpperfmon" on page 20.

Reconfiguring Greenplum Performance Monitor

The configuration for Greenplum Performance Monitor is stored in `$MASTER_DATA_DIRECTORY/gpperfmon/conf/gpperfmon.conf`. This configuration file contains all the configuration options described in Appendix A, "Utility Reference: gpperfmon".

If you want to change selected configuration values, edit `gpperfmon.conf`. If you want to change most or all configuration values for a monitor instance, you can run `gpperfmon -- setup`.

Editing the Configuration File

You can directly edit `gpperfmon.conf` to adjust a single value such as `min_detailed_query_time`.

To edit the configuration file

1. In a text editor, open `$GPHOMEconf/gpperfmon.conf`.
2. Locate the desired parameter and type the new value. For example, to change `min_detailed_query_time`, modify this setting found in the `[GPMMON]` section of the file:

   ```
   min_detailed_query_time=360
   ```
3. When you are finished making changes, save `gpperfmon.conf`.
4. Restart the monitor. For example:

   ```
   $ gpperfmon --restart
   ```

Reconfiguring a Monitor Instance with gpperfmon

Running `gpperfmon --setup` after installation effectively resets all values in `gpperfmon.conf`. This may be useful in situations where you need to change many or all of the values for a monitor instance.

Also, with `gpperfmon --setup` you can replace the existing monitor database with a fresh instance, or change values that are not available in `gpperfmon.conf`, such as the web server port and the DN values used for SSL.

To reconfigure a monitor instance with gpperfmon

1. Run `gpperfmon --setup`.
2. The utility displays the message warning that a monitor configuration file already exists. Type `Y` to reconfigure the monitor instance.
3. The utility prompts you to create a new database, warning you that all existing performance data will be lost. Type `Y` or `N` as desired.
4. Enter a value for Quantum. The default is 15 seconds.
5. Enter a value for Minimum query run time. The default is 60.
6. Enter a value for Minimum detailed query run time. The default value displayed is the value you specified for Minimum query run time.
7. Enter a location for the Greenplum Performance Monitor log files. The default is `$MASTER_DATA_DIRECTORY/gpperfmon/logs`.
8. Enter a value for the maximum number of data points the Web UI can return in one call. The default is 600.
9. The monitoring GUI can highlight specific query or system metrics that exceed a certain threshold. Enter desired values for the following metrics:

   ```
   Load2 threshold [2.00]:
   CPU usage threshold (percent) [80]:
   Memory usage threshold (percent) [80]:
   Disk usage threshold (bytes) [0]:
   ```

```
Network usage threshold (bytes) [10000]:
CPU elapsed threshold [70]:
CPU skew threshold [100]:
```

10. Enter the web server port for the user interface. The default 8080.
11. (Optional) Enter `Y` or `N` to set up SSL connections for the user interface. If you enter `Y`, you are prompted for the following DN information:

    ```
    Country Name (2 letter code) [GB]:
    State or Province Name (full name) [Berkshire]:
    Locality Name (eg, city) [Newbury]:, s
    Organization Name (eg, company) [My Company Ltd]:
    Organizational Unit Name (eg, section) []:
    Common Name (eg, your name or your server's hostname) []:
    Email Address []:
    ```

    When the utility displays a message indicating that `gpperfmon` configuration is complete, you have successfully reconfigured Greenplum Performance Monitor.

Agent Administration

This section describes basic agent administration tasks, including setting up a cronograph job to restart the master agent in case of failure, adding monitor hosts, and viewing agent log files.

Restarting the Master Agent

In this release, the master agent is not automatically restarted if it fails. Greenplum recommends setting up a cronograph job on the master host to start the master agent every few minutes. If the agent is already running, the cron job fails. If the agent is not running, the cron job restarts it.

Adding and Removing Monitored Hosts

Segment agents on new hosts are detected automatically by the master agent. To add a new monitored host, follow the procedure described in "Installing the Segment Monitor Agents" on page 12 . The master agent detects, starts, and begins harvesting data from the segment agent.

Viewing and Maintaining Master Agent Log Files

Log messages for the master agent are written to `$MASTER_DATA_DIRECTORY/gpperfmon/logs/gpmmon.log` by default. To change the log file location, run `gpperfmon -- setup`, or edit the `log_location` parameter in `gpperfmon.conf`. For more information, see Reconfiguring Greenplum Performance Monitor.

If `verbose` is set to the default value of `0`, no messages are logged. To log information for troubleshooting or debugging, set this value to 1 or 2 for detailed messages.

At higher logging levels, the size of the log files may grow dramatically. If the log files grow to an unmanageable size, restart the monitor to clear out existing monitor log messages.

Greenplum Performance Monitor Database Administration

As a distributed Greenplum database, the monitor database interoperates with the set of administration tools provided with your Greenplum installation. This section describes how to use Greenplum utilities in the routine maintenance of the monitor database.

Backing Up and Restoring the Monitor Database

You can back up and restore using the `gpdump` and `gprestore` utilities as described in chapter 21 of the *Greenplum Database Adminstrator Guide*, "Backing Up and Restoring Databases." This document can be found in the `$GPHOME/docs` directory of your Greenplum Database installation, or on the Greenplum Network.

Because the monitor database has a low number of tables, you may prefer to devise a backup plan using the table-level backup features of `gpdump`. For example, you can create scripts to run `gpdump` to back up the monthly partitions of the historical data tables on a monthly schedule. Alternately, you can back up your monitor database at the database level.

Maintaining Historical Data

The history tables are partitioned by month, allowing you to drop partitions for data that you no longer need. The `gpperfmon` installer creates eighteen months of partitioned tables. When that period is over, you may want to create new monthly partitions. For more information on partitioning Greenplum Database tables, see chapter 14 of the *Greenplum Database Adminstrator Guide*, "Defining Database Objects."

Viewing and Maintaining Database Log Files

Web Server Administration

The lighttpd web server is installed in <dir>. For detailed information on lighttpd administration, see http://www.lighttpd.net/.

Starting and Stopping the Web Server

The `gpperfmon` utility starts, stops and restarts the lighttpd web server together with the monitor agents, using the following option flags:

```
--start
```

```
--stop
--restart
```

If you want to start and stop the web server independently, <TBD: from Rui or lighttpd docs>

Web server port

Specifies the port for the web server that provides the user interface and web API.

SSL Setup Options

These options for SSL connections to the user interface are set by `gpperfmon --setup`.

Country Name

Country name in format of a two-letter code. See ISO for a list of valid country codes.

State or Province Name

State or province name.

Locality Name

Locality name, such as city or town name.

Organization Name

Company name.

Organizational Unit Name

Section or function. For example, “system administration.”

Common Name

Domain name. This value requires a server name to be entered for verification purposes. Entering anything other than a domain name yields an invalid certificate.

Email Address

A valid email address.

Viewing and Maintaining Web Server Log Files

Web server log

web.py log

4 Using Greenplum Performance Monitor

<TBD: generalities here on viewing performance data to achieve various goals; there's a GUI and some prepackaged views, etc. Then break the views out under lower headings, or give them their own bucket?>

About the Greenplum Performance Monitor User Interface

Through the Greenplum Performance Monitor UI, you can view different types performance data at multiple levels of granularity. The UI provides a general view of current activity, as well as detailed views of particular queries and query metrics. Using the historical data views, you can analyze historical data for trends and or anomalies in the performance of your Greenplum Database.

Greenplum Performance Monitor user interface is supported for the following browsers with Macromedia Flash:

- Internet Explorer for Windows XP and Vista
- Mozilla Firefox for Windows and Linux
- Apple Safari for Macintosh

The user interface is supported by a lighttpd web server installed on the master host. For more information about the web server, see "Web Server Administration" on page 17.

The web service API for the user interface is installed in <TBD: which dir is the right one?> . For more information about the web service, see <TBD: reference doc>.

Monitoring the Current System Workload

<TBD: describe the right overview pane, with some details on drill-downs>

Monitoring Active Queries

<TBD: describe the left pane, again with drill downs>

Viewing Historical Query Metrics

<TBD: break out the by-time period views and the serious drilldowns into iterator information in this section>

A. Utility Reference: gpperfmon

This utility controls various aspects of the Greenplum Performance Monitor, including configuration and starting and stopping monitor agents.

Synopsis

```
gpperfmon [--setup] [--start] [--stop] [--restart] [--status]
[--version] [--noweb]
```

Description

The `gpperfmon` utility controls various aspects of Greenplum Performance Monitor. This utility configures the monitor components at installation time, starts and stops the monitor, and provides status and version information.

For starting and stopping the monitor, `gpperfmon` controls the master monitor agent, which passes commands to all segment agents. Commands for starting and stopping also control the web server, unless the --noweb options is specified.

Options

`--setup`

Configures monitor components on the master host. With this option, `gpperfmon` prompts for values to configure monitor components and writes the values to `gpperfmon.conf` and `lighttpd.conf`. For more information on these configuration parameters, see Appendix B, "Configuration Reference: gpperfmon.conf".

Running `gpperfmon --setup` after a successful installation effective resets all configuration values to the defaults. Use this option at installation time, or any time when you want to reconfigure all or most monitor parameters.

`--start`

Starts the web server and master agent, which in turn starts all segment monitor agents in the array. Can be used with the flag `--noweb` to start only the monitor agents without starting the web server.

`--stop`

Stops the web server and master agent, which in turn stops all segment monitor agents in the array. Can be used with the flag `--noweb` to stop only the monitor agents without stopping the web server.

`--restart`

Restarts the web server and master agent, which in turn restarts all segment monitor agents in the array. Can be used with the flag `--noweb` to restart only the monitor agents without restarting the web server.

This flag is used with `gpperfmon` to reload configuration files after editing `gpperfmon.conf` or `lighttpd.conf`.

`--status`

Displays the status, either `Running` or `Stopped`, of the monitor and web server.

`--version`

Displays the version of the web server, master agent, and segment monitor agents.

`--noweb`

Used with `--start`, `--stop` and `--restart` to perform the command for the monitor agents only, excluding the web server.

Examples

Start the Greenplum Monitor master agent but not the web server for the user interface:

```
gppperfmon --start --noweb
```

Start the utility in order to install or to change configuration values:

```
gpperfmon --setup
```

Check the status of the monitor and web server:

```
gpperfmon --status
```

See Also

Greenplum Performance Monitor Catalog Reference

B. Configuration Reference: gpperfmon.conf

<TBD: who can set these (only DB owner, superuser?); restart with gpperfmon in all cases?>

Master Agent Options

These options for the master agent process `gpmmon` are set by `gpperfmon --setup`.

min_query_time

Specifies the minimum query run time in seconds for statistics collection. The monitor logs all queries that run longer than this value in the `queries_history` table. For queries with shorter run times, no historical data is collected. Defaults to 60 seconds.

If you know that you want to collect data for all queries, you can set this parameter to a low value. Setting the minimum query run time to zero, however, collects data even for the numerous queries run by the monitor itself, creating a large amount of data that may not be useful.

min_detailed_query_time

Specifies the minimum iterator run time in seconds for statistics collection. The monitor logs all iterators that run longer than this value in the `iterators_history` table. For iterators with shorter run times, no data is collected.

Defaults to the value of `min_query_time`, and must always be equal to, or greater than, the value of **`-m`**. Setting **`-d`** higher than **`-m`** allows you to log detailed iterator data only for especially complex, long-running queries, while still logging basic query data for shorter queries.

Given the complexity and size of iterator data, you may need to adjust this parameter according to the size of data collected. If a cost-benefit evaluation determines that the *iterators_history* tables are growing to excessive size without providing useful information, you can raise the value of this parameter to log iterator detail for fewer queries.

quantum

Specifies the time in seconds between updates from performance monitor agents on all segments. Defaults to 15 seconds, with valid values limited to the range between 10 and 120 seconds.

If you prefer a less granular view of performance, or want to collect and analyze minimal amounts of data for system metrics, choose a higher quantum. <TBD: talk to Ben to verify this>. To collect data more frequently, choose a lower value.

verbose

Verbose mode shows progress and status messages of the performance tests as they are run. The default is `0`, which displays and logs no progress or status messages. Valid values of `1` and `2` display increasingly detailed messages, and are recommended only for troubleshooting purposes. These higher settings cause dramatic growth in log file size.

log_location

Specifies a directory location for monitor log files. Default is `$MASTER_DATA_DIRECTORY/gpperfmon/logs`.

User Interface Options

These options for the user interface and its web server are set by `gpperfmon --setup`.



The related parameter `web server port t`, which is set by `gpperfmon --setup`, resides in `httpd.conf`.

max_data_points

Specifies the maximum number of data points to display in the user interface in order to prevent returning data sets of unmanageable size. Default is 600. When data sets larger than this value are requested, the data is automatically aggregated and reduced to more usable units for display in the user interface.

load2_threshold

Sets the threshold to highlight *load2* metrics in the user interface. The *load2* column reflects the CPU load average for the fifteen-minute period prior to metric collection.

cpu_threshold

Sets the threshold to highlight CPU usage metrics in the user interface.

memory_threshold

Sets the threshold to highlight memory usage metrics in the user interface.

disk_threshold

Sets the threshold to highlight disk usage metrics in the user interface.

network_threshold

Sets the threshold to highlight network throughput metrics in the user interface.

cpu_elapsed_threshold

Sets the threshold to highlight CPU elapsed metrics in the user interface.

skew_cpu_threshold

Sets the threshold to highlight skew metrics in the user interface.

C. Greenplum Performance Monitor Catalog Reference

This is a reference of the system catalog tables for Greenplum Performance Monitor.

The monitor database stores each performance statistic type (system metrics, queries, and iterators) in three separate sets of tables: one for current statistics, one for "tail" data in transition, and one for historical data.

- system_now
- system_tail
- system_history
- database_now
- database_tail
- database_history
- queries_now
- queries_tail
- queries_history
- iterators_now
- iterators_tail
- iterators_history

system_now

The table *system_now* contains current system utilization metrics. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/` . Current system utilization data is stored in *system_now* during the period between data collection from the monitor agents and automatic commitment to the *system_history* table.

Table J.1 gpperfmon.system_now

column	type	description
ctime	timestamp	Time this row was created.
hostname	varchar(64)	Hostname of the system to which the metric belongs.
mem_total	bigint	Total system memory for the system to which the metric belongs, including actual memory and swap space.
mem_used	bigint	Used system memory for the system to which the metric belongs.
mem_actual_used	bigint	Used actual memory (memory not including portion reserved for cache and buffers).
mem_actual_free	bigint	Free actual memory (memory not including portion reserved for cache and buffers).
swap_total	bigint	Total swap space for the system to which the metric belongs.
swap_used	bigint	Used swap space for the system to which the metric belongs.
swap_page_in	bigint	Number of swap pages in.
swap_page_out	bigint	Number of swap pages out.
cpu_user	float	CPU usage by the user.
cpu_sys	float	CPU usage for the system.
cpu_idle	float	Idle CPU capacity at metric collection time.
load0	float	CPU load average for the prior one-minute period.
load1		CPU load average for the prior five-minute period.
load2		CPU load average for the prior fifteen-minute period.
quantum	int	Interval between metric collection for this metric entry.
disk_ro_rate	bigint	Disk read operations per second.
disk_wo_rate	bigint	Disk write operations per second.
disk_rb_rate	bigint	Bytes per second for disk read operations.
disk_wb_rate	bigint	Bytes per second for disk write operations.
net_rp_rate	bigint	Packets per second on the system network for read operations.
net_wp_rate	bigint	Packets per second on the system network for write operations.

Table J.1 gpperfmon.system_now

column	type	description
net_rb_rate	bigint	Bytes per second on the system network for read operations.
net_wb_rate	bigint	Bytes per second on the system network for write operations.

system_tail

The table *system_tail* contains system utilization metrics queued for commitment to *system_history.* This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. System utilization data is stored in *system_tail* between flushing from *system_now* and automatic commitment to *system_history*.

Table J.1 gpperfmon.system_now

column	type	description
ctime	timestamp	Time this row was created.
hostname	varchar(64)	Hostname of the system to which the metric belongs.
mem_total	bigint	Total system memory for the system to which the metric belongs, including actual memory and swap space.
mem_used	bigint	Used system memory for the system to which the metric belongs.
mem_actual_used	bigint	Used actual memory (memory not including portion reserved for cache and buffers).
mem_actual_free	bigint	Free actual memory (memory not including portion reserved for cache and buffers).
swap_total	bigint	Total swap space for the system to which the metric belongs.
swap_used	bigint	Used swap space for the system to which the metric belongs.
swap_page_in	bigint	Number of swap pages in.
swap_page_out	bigint	Number of swap pages out.
cpu_user	float	CPU usage by the user.
cpu_sys	float	CPU usage for the system.
cpu_idle	float	Idle CPU capacity at metric collection time.
load0	float	CPU load average for the prior one-minute period.
load1		CPU load average for the prior five-minute period.
load2		CPU load average for the prior fifteen-minute period.
quantum	int	Interval between metric collection for this metric entry.
disk_ro_rate	bigint	Disk read operations per second.
disk_wo_rate	bigint	Disk write operations per second.
disk_rb_rate	bigint	Bytes per second for disk read operations.
disk_wb_rate	bigint	Bytes per second for disk write operations.
net_rp_rate	bigint	Packets per second on the system network for read operations.
net_wp_rate	bigint	Packets per second on the system network for write operations.

Table J.1 gpperfmon.system_now

column	type	description
net_rb_rate	bigint	Bytes per second on the system network for read operations.
net_wb_rate	bigint	Bytes per second on the system network for write operations.

system_history

The table *system_history* stores historical system utilization metrics. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need.

Table J.1 gpperfmon.system_history

column	type	description
ctime	timestamp	Time this row was created.
hostname	varchar (64)	Hostname of the system to which the metric belongs.
mem_total	bigint	Total system memory for the system to which the metric belongs.
mem_used	bigint	Used system memory for the system to which the metric belongs.
mem_actual_used	bigint	Used actual memory (memory not including portion reserved for cache and buffers).
mem_actual_free	bigint	Free actual memory (memory not including portion reserved for cache and buffers).
swap_total	bigint	Total swap space for the system to which the metric belongs.
swap_used	bigint	Used swap space for the system to which the metric belongs.
swap_page_in	bigint	Number of swap pages in.
swap_page_out	bigint	Number of swap pages out.
cpu_user	float	CPU usage by the user.
cpu_sys	float	CPU usage for the system.
cpu_idle	float	Idle CPU capacity at metric collection time.
load0	float	CPU load average for the prior one-minute period.
load1	float	CPU load average for the prior five-minute period.
load2	float	CPU load average for the prior fifteen-minute period.
quantum	int	Interval between metric collection for this metric entry.
disk_ro_rate	bigint	Disk read operations per second.
disk_wo_rate	bigint	Disk write operations per second.
disk_rb_rate	bigint	Bytes per second for disk read operations.
disk_wb_rate	bigint	Bytes per second for disk write operations.
net_rp_rate	bigint	Packets per second on the system network for read operations.
net_wp_rate	bigint	Packets per second on the system network for write operations.

Table J.1 gpperfmon.system_history

column	type	description
net_rb_rate	bigint	Bytes per second on the system network for read operations.
net_wb_rate	bigint	Bytes per second on the system network for write operations.

database_now

The table *database_now* contains database-wide query information for current queries. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current database-wide query data is stored in *database_now* for each quantum period, after which is passes to *database_tail*..

Table J.1 gpperfmon.iterators_history

column	type	description
ctime	timestamp	Time this row was created.
queries_total	int	The total number of queries.
queries_running	int	The number of queries currently running.
queries_queued	int	The number of queries queued.

database_tail

The table *database_tail* contains database-wide query information for current queries. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current database-wide query data is stored in *database_tail* between flushing from *database_now* and automatic commitment to the *database_history* table..

Table J.1 gpperfmon.iterators_history

column	type	description
ctime	timestamp	Time this row was created.
queries_total	int	The total number of queries.
queries_running	int	The number of queries currently running.
queries_queued	int	The number of queries queued.

database_history

The table *database_history* contains historical database-wide query information. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need..

Table J.1 gpperfmon.iterators_history

column	type	description
ctime	timestamp	Time this row was created.
queries_total	int	The total number of queries.
queries_running	int	The number of queries running at data collection time for this row.
queries_queued	int	The number of queries queued at data collection time for this row.

queries_now

The table *queries_now* contains currently running query status information. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current query status data is stored in *quieries_now* until the query is completed, refreshing every quantum period. When the query is completed, the query data is passed into *queries_tail*.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the query data tables.

Table J.1 gpperfmon.queries_now

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All records associated with the query will have the same `ccnt`.
uname	varchar(64)	Username that issued this query.
db	varchar(64)	Name of the database queried.
tsubmit	timestamp	Time the query was submitted.
tstart	timestamp	Time the query was started.
tfin	timestamp	Time the query finished. For running queries, this value is always zero.
status	varchar(64)	Status of the query -- `start`, `done`, or `abort`.
rows_out	bigin	Rows out for the query.
cpu_elapsed	bigin	CPU usage for this query across all segments. This is the sum of the CPU usage values taken from all segments in the database array.
cpu_currpct	float	Current CPU percent average for all processes executing this query. The percentages for all processes running on each segment are averaged, and then the average of all those values is calculated to render this metric.
skew_cpu	float	Coefficient of variation for `cpu_elapsed` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
skew_rows	float	Coefficient of variation for `rows_in` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
query_hash	bigint	This column is not implemented at time of writing.
query_text	text	Query text for this query.
query_plan	text	This column is not implemented at time of writing.

queries_tail

The table *queries_tail* contains currently running query status information. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/` . Current query status data is stored in *queries_tail* between flushing from *queries_now* and automatic commitment to the *queries_history* table.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the query data tables.

Table J.1 gpmon_2.queries_now

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All records associated with the query will have the same `ccnt`.
uname	varchar(64)	Username that issued this query.
db	varchar(64)	Name of the database queried.
tsubmit	timestamp	Time the query was submitted.
tstart	timestamp	Time the query was started.
tfin	timestamp	Time the query finished. For running queries, this value is always zero.
status	varchar(64)	Status of the query -- `start`, `done`, or `abort`.
rows_out	bigin	Rows out for the query.
cpu_elapsed	bigin	CPU usage for this query across all segments. This is the sum of the CPU usage values taken from all segments in the database array.
cpu_currpct	float	Current CPU percent average for all processes executing this query. The percentages for all processes running on each segment are averaged, and then the average of all those values is calculated to render this metric.
skew_cpu	float	Coefficient of variation for `cpu_elapsed` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
skew_rows	float	Coefficient of variation for `rows_in` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
query_hash	bigint	This column is not implemented at time of writing.
query_text	text	Query text for this query.
query_plan	text	This column is not implemented at time of writing.

queries_history

The table *queries_history* stores historical query list status. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the query data tables.

Table J.1 gpperfmon.queries_history

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All records associated with the query will have the same `ccnt`.
username	varchar(64)	Username that issued this query.
db	varchar(64)	Name of the database queried.
tsubmit	timestamp	Time the query was submitted.
tstart	timestamp	Time the query was started.
tfinish	timestamp	Time the query finished.
status	varchar(64)	Status of the query -- `start`, `done`, or `abort`.
rows_out	bigin	Rows out for the query.
cpu_elapsed	bigin	CPU usage for this query across all segments. This is the sum of the CPU usage values taken from all segments in the database array.
cpu_currpct	float	Current CPU percent average is always zero in historical data.
skew_cpu	float	Coefficient of variation for `cpu_elapsed` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
skew_rows	float	Coefficient of variation for `rows_in` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95.
query_hash	bigint	This column is not implemented at time of writing.
query_text	text	Query text for this query.
query_plan	text	This column is not implemented at time of writing.

iterators_now

The table *iterators_now* contains current query iterators and their metrics. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current iterator data is stored in *iterators_now* until the query is completed, refreshing every quantum period. When the query is completed, the detailed iterator data is passed into *iterators_tail*.

A query iterator refers to a node or operation in a query plan. For example, a sequential scan operation may be one node or iterator in a particular query plan. By looking at the metrics of the individual operations in a query plan, you can determine the status of a query's execution.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the iterator data tables.

Table J.1 gpperfmon.iterators_now

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All iterator records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All iterator records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All iterator records associated with the query will have the same `ccnt`.
segid	int	The segment ID (`dbid` from `gp_configuration`).
pid	int	The process ID of the `postgres` process executing this iterator.
nid	int	The ID of this query plan node from the slice plan.
pnid	int	The ID of the parent query plan node from the slice plan.
hostname	varchar(64)	Hostname for the system where the iterator is executed.
ntype	varchar(64)	The query plan node operation type. Possible values are: `AdaptiveNestLoopJoin`, `HashAgg`, `Append`, `AppendOnlyScan`, `BitmapAnd`, `BitmapHeapScan`, `BitmapIndexScan`, `BitmapOr`, `ExternalScan`, `FunctionScan`, `Group`, `Hash`, `HashJoin`, `IndexScan`, `Limit`, `Materialize`, `MergeJoin`, `Motion`, `NestLoopJoin`, `ResultNode`, `SequentialScan`, `SetOpNode`, `ShareInputScan`, `Sort`, `SubqueryScan`, `TidScan`, `Unique`, `ValuesScan`, `Window`.
nstatus	varchar(64)	The status of this query plan node operation. Possible values are: `silent`, `submit`, `start`, `done`, or `abort`.
tstart	timestamp	Start time for the iterator.
tduration	int	Duration of the execution.
pmemsize	bigint	Memory size from Greenplum Database memory allocation data.

Table J.1 gpperfmon.iterators_now

column	type	description
pmemmax	bigint	Maximum memory from Greenplum Database memory allocation data.
memsize	bigint	Memory size from OS memory allocation data.
memresid	bigint	Resident memory, as opposed to shared or virtual memory.
memshare	bigint	Shared memory figure from OS memory allocation data.
cpu_elapsed	bigint	Total CPU usage of the process executing the iterator.
cpu_currpct	float	CPU percentage for the process executing the iterator.
phase	varchar(64)	The iterator phase.
rowsout	bigint	Number of rows output by the iterator.
rowsout_est	bigint	The query planner's estimate of rows output by the iterator.
m0_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. For all operations, this metric name is `Rows In`.
m0_unit	varchar(64)	The unit of measure for this metric. For all operations (ntype), this unit of measure is `Rows`.
m0_val	bigint	The value of this metric.
m0_est	bigint	The estimated value of this metric.

m1_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Tuples`
		`Append - Append Current Input Source`
		`AppendOnlyScan - AppendOnlyScan Rescan`
		`BitmapHeapScan - BitmapHeapScan Pages`
		`Hash - Hash Spill Batches`
		`HashJoin - HashJoin Spill Batches`
		`IndexScan - IndexScan Restore Pos`
		`MergeJoin - MergeJoin Inner Side Tuples`
		`NestLoopJoin - NestLoopJoin Inner Tuples`
		`SequentialScan - SeqScan Page Stats`
		`ShareInputScan - ShareInputScan Restore Pos`
		`Sort - Sort Memory Usage`
		`SubqueryScan - SubqueryScan Rescan`

Table J.1 gpperfmon.iterators_now

column	type	description

m1_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Append - Inputs`
		`AppendOnlyScan - Rescans`
		`BitmapHeapScan - Pages`
		`Hash - Batches`
		`HashJoin - Batches`
		`IndexScan - Restores`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Pages`
		`ShareInputScan - Restores`
		`Sort - Bytes`
		`SubqueryScan - Rescans`

m1_val	bigint	The value of this metric.
m1_est	bigint	The estimated value of this metric.

m2_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Bytes`
		`Hash - Hash Spill Tuples`
		`HashJoin - HashJoin Spill Tuples`
		`IndexScan - IndexScan Rescan`
		`MergeJoin - MergeJoin Outer Side Tuples`
		`NestLoopJoin - NestLoopJoin Outer Tuples`
		`SequentialScan - SeqScan Restore Pos`
		`ShareInputScan - ShareInputScan Rescan`
		`Sort - Sort Spill Tuples`

m2_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Hash - Tuples`
		`HashJoin - Tuples`
		`IndexScan - Rescans`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Restores`
		`ShareInputScan - Rescans`
		`Sort - Tuples`

m2_val	bigint	The value of this metric.
m2_est	bigint	The estimated value of this metric.

Table J.1 gpperfmon.iterators_now

column	type	description

m3_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Batches`
		`Hash - Hash Spill Bytes`
		`HashJoin - HashJoin Spill Bytes`
		`SequentialScan - SeqScan Rescan`
		`Sort - Sort Spill Bytes`

m3_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Batches`
		`Hash - Bytes`
		`HashJoin - Bytes`
		`SequentialScan - Rescans`
		`Sort - Bytes`

m3_val	bigint	The value of this metric.
m3_est	bigint	The estimated value of this metric.

m4_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Pass`
		`Sort - Sort Spill Pass`

m4_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Passes`
		`Sort - Passes`

m4_val	bigint	The value of this metric.
m4_est	bigint	The estimated value of this metric.

m5_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Tuples`
		`Sort - Sort Current Spill Pass Tuples`

m5_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Sort - Tuples`

m5_val	bigint	The value of this metric.
m5_est	bigint	The estimated value of this metric.

m6_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Bytes`
		`Sort - Sort Current Spill Pass Bytes`

Table J.1 gpperfmon.iterators_now

column	type	description

m6_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Sort - Bytes`

m6_val	bigint	The value of this metric.
m6_est	bigint	The estimated value of this metric.
m7_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Tuples`
m7_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Tuples`
m7_val	bigint	The value of this metric.
m7_est	bigint	The estimated value of this metric.
m8_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Bytes`
m8_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Bytes`
m8_val	bigint	The value of this metric.
m8_est	bigint	The estimated value of this metric.
m9_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Batch`
m9_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Batches`
m9_val	bigint	The value of this metric.
m9_est	bigint	The estimated value of this metric.

iterators_tail

The table *iterators_tail* contains current query iterators and their metrics. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/` . Current iterator data is stored in *iterators_tail* between flushing from *iterators_now* and automatic commitment to *iterators_history*.

A query iterator refers to a node or operation in a query plan. For example, a sequential scan operation may be one node or iterator in a particular query plan. By looking at the metrics of the individual operations in a query plan, you can determine the status of a query's execution.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the iterator data tables.

Table J.1 gpmon_2.iterators_now

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All iterator records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All iterator records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All iterator records associated with the query will have the same `ccnt`.
segid	int	The segment ID (`dbid` from `gp_configuration`).
pid	int	The process ID of the `postgres` process executing this iterator.
nid	int	The ID of this query plan node from the slice plan.
pnid	int	The ID of the parent query plan node from the slice plan.
hostname	varchar(64)	Hostname for the system where the iterator is executed.
ntype	varchar(64)	The query plan node operation type. Possible values are: `AdaptiveNestLoopJoin`, `HashAgg`, `Append`, `AppendOnlyScan`, `BitmapAnd`, `BitmapHeapScan`, `BitmapIndexScan`, `BitmapOr`, `ExternalScan`, `FunctionScan`, `Group`, `Hash`, `HashJoin`, `IndexScan`, `Limit`, `Materialize`, `MergeJoin`, `Motion`, `NestLoopJoin`, `ResultNode`, `SequentialScan`, `SetOpNode`, `ShareInputScan`, `Sort`, `SubqueryScan`, `TidScan`, `Unique`, `ValuesScan`, `Window`.
nstatus	varchar(64)	The status of this query plan node operation. Possible values are: `silent`, `submit`, `start`, `done`, or `abort`.
tstart	timestamp	Start time for the iterator.
tduration	int	Duration of the execution.
pmemsize	bigint	Memory size from Greenplum Database memory allocation data.
pmemmax	bigint	Maximum memory from Greenplum Database memory allocation data.

Table J.1 gpmon_2.iterators_now

column	type	description
memsize	bigint	Memory size from OS memory allocation data.
memresid	bigint	Resident memory, as opposed to shared or virtual memory.
memshare	bigint	Shared memory figure from OS memory allocation data.
cpu_elapsed	bigint	Total CPU usage of the process executing the iterator.
cpu_currpct	float	CPU percentage for the process executing the iterator.
phase	varchar(64)	The iterator phase.
rowsout	bigint	Number of rows output by the iterator.
rowsout_est	bigint	The query planner's estimate of rows output by the iterator.
m0_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. For all operations, this metric name is `Rows In`.
m0_unit	varchar(64)	The unit of measure for this metric. For all operations (ntype), this unit of measure is `Rows`.
m0_val	bigint	The value of this metric.
m0_est	bigint	The estimated value of this metric.

m1_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Tuples`
		`Append - Append Current Input Source`
		`AppendOnlyScan - AppendOnlyScan Rescan`
		`BitmapHeapScan - BitmapHeapScan Pages`
		`Hash - Hash Spill Batches`
		`HashJoin - HashJoin Spill Batches`
		`IndexScan - IndexScan Restore Pos`
		`MergeJoin - MergeJoin Inner Side Tuples`
		`NestLoopJoin - NestLoopJoin Inner Tuples`
		`SequentialScan - SeqScan Page Stats`
		`ShareInputScan - ShareInputScan Restore Pos`
		`Sort - Sort Memory Usage`
		`SubqueryScan - SubqueryScan Rescan`

Table J.1 gpmon_2.iterators_now

column	type	description

m1_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Append - Inputs`
		`AppendOnlyScan - Rescans`
		`BitmapHeapScan - Pages`
		`Hash - Batches`
		`HashJoin - Batches`
		`IndexScan - Restores`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Pages`
		`ShareInputScan - Restores`
		`Sort - Bytes`
		`SubqueryScan - Rescans`

m1_val	bigint	The value of this metric.
m1_est	bigint	The estimated value of this metric.

m2_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Bytes`
		`Hash - Hash Spill Tuples`
		`HashJoin - HashJoin Spill Tuples`
		`IndexScan - IndexScan Rescan`
		`MergeJoin - MergeJoin Outer Side Tuples`
		`NestLoopJoin - NestLoopJoin Outer Tuples`
		`SequentialScan - SeqScan Restore Pos`
		`ShareInputScan - ShareInputScan Rescan`
		`Sort - Sort Spill Tuples`

m2_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Hash - Tuples`
		`HashJoin - Tuples`
		`IndexScan - Rescans`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Restores`
		`ShareInputScan - Rescans`
		`Sort - Tuples`

m2_val	bigint	The value of this metric.
m2_est	bigint	The estimated value of this metric.

Table J.1 gpmon_2.iterators_now

column	type	description

m3_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Batches`
		`Hash - Hash Spill Bytes`
		`HashJoin - HashJoin Spill Bytes`
		`SequentialScan - SeqScan Rescan`
		`Sort - Sort Spill Bytes`

m3_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Batches`
		`Hash - Bytes`
		`HashJoin - Bytes`
		`SequentialScan - Rescans`
		`Sort - Bytes`

m3_val	bigint	The value of this metric.
m3_est	bigint	The estimated value of this metric.

m4_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Pass`
		`Sort - Sort Spill Pass`

m4_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Passes`
		`Sort - Passes`

m4_val	bigint	The value of this metric.
m4_est	bigint	The estimated value of this metric.

m5_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Tuples`
		`Sort - Sort Current Spill Pass Tuples`

m5_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Sort - Tuples`

m5_val	bigint	The value of this metric.
m5_est	bigint	The estimated value of this metric.

m6_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Bytes`
		`Sort - Sort Current Spill Pass Bytes`

Table J.1 gpmon_2.iterators_now

column	type	description

m6_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Sort - Bytes`

m6_val	bigint	The value of this metric.
m6_est	bigint	The estimated value of this metric.
m7_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Tuples`
m7_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Tuples`
m7_val	bigint	The value of this metric.
m7_est	bigint	The estimated value of this metric.
m8_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Bytes`
m8_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Bytes`
m8_val	bigint	The value of this metric.
m8_est	bigint	The estimated value of this metric.
m9_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Batch`
m9_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Batches`
m9_val	bigint	The value of this metric.
m9_est	bigint	The estimated value of this metric.

iterators_history

The table *iterators_history* contains historical query iterators and their metrics. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need.

A query iterator refers to a node or operation in a query plan. For example, a sequential scan operation may be one node or iterator in a particular query plan.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the iterator data tables.

Table J.1 gpperfmon.iterators_history

column	type	description
ctime	timestamp	Time this row was created.
tmid	int	A time identifier for a particular query. All iterator records associated with the query will have the same `tmid`.
ssid	int	The session id as shown by `gp_session_id`. All iterator records associated with the query will have the same `ssid`.
ccnt	in	The command number within this session as shown by `gp_command_count`. All iterator records associated with the query will have the same `ccnt`.
segid	int	The segment ID (`dbid` from `gp_configuration`).
pid	int	The process ID of the `postgres` process executing this iterator.
nid	int	The ID of this query plan node from the slice plan.
pnid	int	The ID of the parent query plan node from the slice plan.
hostname	varchar(64)	Hostname for the system where the iterator is executed.
ntype	varchar(64)	The query plan node operation type. Possible values are: `AdaptiveNestLoopJoin`, `HashAgg`, `Append`, `AppendOnlyScan`, `BitmapAnd`, `BitmapHeapScan`, `BitmapIndexScan`, `BitmapOr`, `ExternalScan`, `FunctionScan`, `Group`, `Hash`, `HashJoin`, `IndexScan`, `Limit`, `Materialize`, `MergeJoin`, `Motion`, `NestLoopJoin`, `ResultNode`, `SequentialScan`, `SetOpNode`, `ShareInputScan`, `Sort`, `SubqueryScan`, `TidScan`, `Unique`, `ValuesScan`, `Window`.
nstatus	varchar(64)	The status of this query plan node operation. Possible values are: `silent`, `submit`, `start`, `done`, or `abort`.
tstart	timestamp	Start time for the iterator.
tduration	int	Duration of the execution.
pmemsize	bigint	Memory size from Greenplum Database memory allocation data.
pmemmax	bigint	Maximum memory from Greenplum Database memory allocation data.

Table J.1 gpperfmon.iterators_history

column	type	description
memsize	bigint	Memory size from OS memory allocation data.
memresid	bigint	Resident memory, as opposed to shared or virtual memory.
memshare	bigint	Shared memory figure from OS memory allocation data.
cpu_elapsed	bigint	Total CPU usage of the process executing the iterator.
cpu_currpct	float	This value is always zero for historical iterators.
phase	varchar(64)	The iterator phase.
rowsout	bigint	Number of rows output by the iterator.
rowsout_est	bigint	The query planner's estimate of rows output by the iterator.
m0_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. For all operations, this metric name is `Rows In`.
m0_unit	varchar(64)	The unit of measure for this metric. For all operations (ntype), this unit of measure is `Rows`.
m0_val	bigint	The value of this metric.
m0_est	bigint	The estimated value of this metric.

m1_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Tuples`
		`Append - Append Current Input Source`
		`AppendOnlyScan - AppendOnlyScan Rescan`
		`BitmapHeapScan - BitmapHeapScan Pages`
		`Hash - Hash Spill Batches`
		`HashJoin - HashJoin Spill Batches`
		`IndexScan - IndexScan Restore Pos`
		`MergeJoin - MergeJoin Inner Side Tuples`
		`NestLoopJoin - NestLoopJoin Inner Tuples`
		`SequentialScan - SeqScan Page Stats`
		`ShareInputScan - ShareInputScan Restore Pos`
		`Sort - Sort Memory Usage`
		`SubqueryScan - SubqueryScan Rescan`

Table J.1 gpperfmon.iterators_history

column	type	description

m1_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Append - Inputs`
		`AppendOnlyScan - Rescans`
		`BitmapHeapScan - Pages`
		`Hash - Batches`
		`HashJoin - Batches`
		`IndexScan - Restores`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Pages`
		`ShareInputScan - Restores`
		`Sort - Bytes`
		`SubqueryScan - Rescans`

m1_val	bigint	The value of this metric.
m1_est	bigint	The estimated value of this metric.

m2_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Bytes`
		`Hash - Hash Spill Tuples`
		`HashJoin - HashJoin Spill Tuples`
		`IndexScan - IndexScan Rescan`
		`MergeJoin - MergeJoin Outer Side Tuples`
		`NestLoopJoin - NestLoopJoin Outer Tuples`
		`SequentialScan - SeqScan Restore Pos`
		`ShareInputScan - ShareInputScan Rescan`
		`Sort - Sort Spill Tuples`

m2_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Hash - Tuples`
		`HashJoin - Tuples`
		`IndexScan - Rescans`
		`MergeJoin - Tuples`
		`NestLoopJoin - Tuples`
		`SequentialScan - Restores`
		`ShareInputScan - Rescans`
		`Sort - Tuples`

m2_val	bigint	The value of this metric.
m2_est	bigint	The estimated value of this metric.

Table J.1 gpperfmon.iterators_history

column	type	description

m3_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Batches`
		`Hash - Hash Spill Bytes`
		`HashJoin - HashJoin Spill Bytes`
		`SequentialScan - SeqScan Rescan`
		`Sort - Sort Spill Bytes`

m3_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Batches`
		`Hash - Bytes`
		`HashJoin - Bytes`
		`SequentialScan - Rescans`
		`Sort - Bytes`

m3_val	bigint	The value of this metric.
m3_est	bigint	The estimated value of this metric.

m4_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Total Spill Pass`
		`Sort - Sort Spill Pass`

m4_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Passes`
		`Sort - Passes`

m4_val	bigint	The value of this metric.
m4_est	bigint	The estimated value of this metric.

m5_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Tuples`
		`Sort - Sort Current Spill Pass Tuples`

m5_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Tuples`
		`Sort - Tuples`

m5_val	bigint	The value of this metric.
m5_est	bigint	The estimated value of this metric.

m6_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:
		`HashAgg - Agg Current Spill Pass Read Bytes`
		`Sort - Sort Current Spill Pass Bytes`

Table J.1 gpperfmon.iterators_history

column	type	description

m6_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:
		`HashAgg - Bytes`
		`Sort - Bytes`

m6_val	bigint	The value of this metric.
m6_est	bigint	The estimated value of this metric.
m7_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Tuples`
m7_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Tuples`
m7_val	bigint	The value of this metric.
m7_est	bigint	The estimated value of this metric.
m8_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Bytes`
m8_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Bytes`
m8_val	bigint	The value of this metric.
m8_est	bigint	The estimated value of this metric.
m9_name	varchar(64)	Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation: `HashAgg - Agg Current Spill Pass Batch`
m9_unit	varchar(64)	The unit of measure for this metric. This unit is as follows for the corresponding plan node operation: `HashAgg - Batches`
m9_val	bigint	The value of this metric.
m9_est	bigint	The estimated value of this metric.

Index

A

agent
  gpmmon: 6
  gpsmon: 6
architecture: 5

B

bashrc: 10

C

components: 5, 10, 20
configuration: 10, 20
  Greenplum Database: 9
Country Name: 18
cpu_elapsed_threshold: 23
cpu_threshold: 23

D

data analysis: 7
data tables
  current data: 6
  historical: 7
  tail: 7
database
  monitor database: 6
disable monitor: 12
disk_threshold: 23
DN: 15

E

enable monitor: 9

F

files, installer: 9

G

gp_enable_gpperfmon: 9
gp_external_enable_exec: 9
gpmmon: 22
gpmon user: 8
gpperfmon.conf: 10, 14, 20
gpperfmon_port: 9
graphical user interface: 7
Greenplum Database configuration: 9
GUI: 7

H

historical data: 7

I

installation path: 9, 10, 12
installing
  on master host: 9
  segment host: 12
interconnect: 5

L

lighttpd.conf: 10, 20
load2_threshold: 23
log_location: 23

M

max_data_points: 23
memory_threshold: 23
min_detailed_query_time: 22
min_query_time: 22
monitor user: 8

N

network_threshold: 23

P

partitioning: 7
path, installlation: 9
port: 9
  web server: 15
postgresql.conf: 9
preinstallation: 8
process
  gpmmon: 6
  gpsmon: 6
psql: 6

Q

quantum: 6, 22

R

reconfiguring: 14
removing the monitor: 12
role, gpmon: 8

S

segment agent installation: 12
setup: 14
skew_cpu_threshold: 24
SSL: 7, 15
superuser: 8
system requirements: 8

T

tail tables: 7

U

uninstalling: 12
user interface: 7

V

verbose: 23

W

web server: 7
Web server port: 18
web server port: 15